

MyBatisPlus(MP)

鸣谢：黑马程序员。（视频链接：【黑马程序员SSM框架教程_Spring+SpringMVC+Maven高级+SpringBoot+MyBatisPlus企业实用开发技术】https://www.bilibili.com/video/BV1Fi4y1S7ix?vd_source=b7f14ba5e783353d06a99352d23ebca9）



MyBatisPlus(MP)

一、入门

1.入门案例

2.概述

二、标准数据层开发

1.标准CRUD开发

1.1 MP接口

1.2 代码示例

2.分页功能

2.1 配置MP拦截器类，添加MP的分页拦截器

2.2 在 `application.yml` 中开启MP的控制台日志

2.3 执行分页查询

2.4 运行结果

三、条件查询

1.条件查询方式

1.1 单条件查询

1.2 多条件查询（支持链式编程）

1.3 NULL值处理

2.查询投影

2.1 查询结果包含实体类中部分属性

2.2 查询结果包含聚合函数

2.3 假如2.2不用MP，而用MyBatis+原生SQL

3.查询条件设定

4.字段映射与表名映射——@TableField + @TableName

4.1 表字段与实体类属性设计不同步

4.2 实体类中添加了表中未定义的字

4.3 采用 `select *` 查询开放了过多的字段查看权限，存在安全隐患

4.4 表名与实体类名设计不同步

四、DML控制

五、快速开发

一、入门

1.入门案例

1. 创建新模块，选择SpringBoot，并配置模块相关基础信息。
2. 选择当前模块需要使用的技术栈，仅勾选 `MySQL Driver`。
3. 手动添加MP和Druid的起步依赖：

```
1 <dependency>
2   <groupId>com.baomidou</groupId>
3   <artifactId>mybatis-plus-spring-boot3-starter</artifactId>
4   <!-- 使用与SpringBoot3.5.9版本兼容的MP版本 -->
5   <version>3.5.11</version> <!-- 示例版本 -->
6 </dependency>
7 <dependency>
8   <groupId>com.baomidou</groupId>
9   <artifactId>mybatis-plus-jsqlparser</artifactId>
```

```

10     <version>3.5.11</version> <!-- 必须与上面的版本保持一致 -->
11 </dependency>
12 <dependency>
13     <groupId>com.alibaba</groupId>
14     <artifactId>druid</artifactId>
15     <version>1.2.23</version> <!-- 确保支持Spring3.5.9 -->
16 </dependency>

```

4. 在 `application.yml` 文件中设置数据源：

```

1 spring:
2   datasource:
3     type: com.alibaba.druid.pool.DruidDataSource
4     driver-class-name: com.mysql.cj.jdbc.Driver
5     url: jdbc:mysql://localhost:3306/mybatis_plus_db
6     username: root
7     password: 123456

```

5. 根据数据库中要操作的表结构制作实体类 `User`：类名与表名对应，属性名与字段名对应。

6. 定义dao接口，继承 `BaseMapper<User>`：

```

1 @Mapper
2 public interface UserDao extends BaseMapper<User> {
3 }

```

7. 在测试类中注入dao接口，测试功能：

```

1 @SpringBootTest
2 class ApplicationTests {
3
4     @Autowired
5     private UserDao userDao;
6
7     @Test
8     void testGetAll() {
9         List<User> users = userDao.selectList(null);
10        System.out.println(users);
11    }
12 }

```

2.概述

- MyBatisPlus（简称MP）是基于MyBatis框架开发的增强型工具，旨在简化开发、提高效率。
- 官网：<https://baomidou.com>或<https://mybatis.plus>
- 特性：
 - **无侵入**：只做增强不做修改，不会对原有工程产生影响。
 - **损耗小**：启动即会自动注入基本CRUD，性能基本无损耗，直接面向对象操作。
 - **强大的CRUD操作**：内置通用Mapper和通用Service，只需少量手动配置即可实现单表大部分CRUD操作。
 - **支持Lambda**：编写查询条件时，无需担心字段写错。
 - 支持主键自动生成。
 - 内置分页插件。
 -

二、标准数据层开发

1.标准CRUD开发

1.1 MP接口

功能	MP接口
新增	<code>int insert(T t)</code>
删除	<code>int deleteById(Serializable id)</code>
修改	<code>int updateById(T t)</code>
根据id查询	<code>T selectById(Serializable id)</code>
查询全部	<code>List<T> selectList()</code>

功能	MP接口
分页查询	<code>IPage<T> selectPage(IPage<T> page)</code>
条件查询	<code>IPage<T> selectPage(Wrapper<T> queryWrapper)</code>

1.2 代码示例

```

1  @SpringBootTest
2  class Mp01QuickstartApplicationTests {
3
4      @Autowired
5      private UserDao userDao;
6
7      @Test
8      void testInsert() {
9          User user = new User("zsh", "666999", 18, "842951084");
10         userDao.insert(user);
11     }
12
13     @Test
14     void testDelete() {
15         userDao.deleteById(2006633799126016002L);
16     }
17
18     @Test
19     void testUpdate() {
20         User user = new User();
21         user.setId(1L);
22         user.setName("Tommy");
23
24         // 提供哪些字段的新值，MP就只会修改这些字段
25         userDao.updateById(user);
26     }
27
28     @Test
29     void testGetById() {
30         User user = userDao.selectById(2L);
31         System.out.println(user);
32     }
33
34     @Test

```

```
35     void testGetAll() {
36         List<User> users = userDao.selectList(null);
37         System.out.println(users);
38     }
39
40 }
```

2. 分页功能

2.1 配置MP拦截器类，添加MP的分页拦截器

```
1  package com.zsh.config;
2
3  import
4  com.baomidou.mybatisplus.extension.plugins.MybatisPlusInterceptor;
5  import
6  com.baomidou.mybatisplus.extension.plugins.inner.PaginationInnerInterce
7  ptor;
8  import org.springframework.context.annotation.Bean;
9  import org.springframework.context.annotation.Configuration;
10
11 // 配置MP的拦截器
12 @Configuration
13 public class MpConfig {
14
15     @Bean
16     public MybatisPlusInterceptor mpInterceptor() {
17         MybatisPlusInterceptor mpInterceptor = new
18         MybatisPlusInterceptor();
19         // 添加MP的分页拦截器
20         mpInterceptor.addInnerInterceptor(new
21         PaginationInnerInterceptor());
22         return mpInterceptor;
23     }
24 }
```

2.2 在 `application.yml` 中开启MP的控制台日志

```
1 # 开启MP的控制台日志
2 mybatis-plus:
3   configuration:
4     log-impl: org.apache.ibatis.logging.stdout.StdoutImpl
```

2.3 执行分页查询

```
1 @Test
2 void testGetByPage() {
3     // 首先要配置好MP的分页拦截器——config/MpConfig
4     IPage page = new Page(2, 3); // 查第2页，每页显示3条记录
5     userDao.selectPage(page, null);
6     System.out.println("当前页码值: " + page.getCurrent());
7     System.out.println("每页记录数: " + page.getSize());
8     System.out.println("总页数: " + page.getPages());
9     System.out.println("总记录数: " + page.getTotal());
10    System.out.println("page data: " + page.getRecords());
11 }
```

2.4 运行结果

✓ Tests passed: 1 of 1 test – 1 sec 136 ms

```
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
Creating a new SqlSession
SqlSession [org.apache.ibatis.session.defaults.DefaultSqlSession@79ab97fd] was not registered for synchronization because synchronization is not active
2026-01-01T16:33:21.535+08:00 INFO 8708 --- [main] com.alibaba.druid.pool.DruidDataSource : {dataSource-1} inited
JDBC Connection [com.alibaba.druid.pool.DruidStatementConnection@7d3bf8db] will not be managed by Spring
==> Preparing: SELECT COUNT(*) AS total FROM user
==> Parameters:
<== Columns: total
<== Row: 4
<== Total: 1
==> Preparing: SELECT id,name,password,age,tel FROM user LIMIT ?,?
==> Parameters: 3(Long), 3(Long)
<== Columns: id, name, password, age, tel
<== Row: 4, 黑马, itcast, 15, 4006184000
<== Total: 1
Closing non transactional SqlSession [org.apache.ibatis.session.defaults.DefaultSqlSession@79ab97fd]
当前页码值: 2
每页记录数: 3
总页数: 2
总记录数: 4
page data: [User(id=4, name=黑马, password=itcast, age=15, tel=4006184000)]
```

三、条件查询

1.条件查询方式

1.1 单条件查询

```
1  @SpringBootTest
2  class Mp02ConditionQueryApplicationTests {
3
4      @Autowired
5      private UserDao userDao;
6
7      @Test
8      void test1() {
9          // 方式一：按条件查询
10         QueryWrapper qw = new QueryWrapper();
11         qw.lt("age", 18);
12         List<User> users = userDao.selectList(qw);
13         System.out.println(users);
14     }
15
16     @Test
17     void test2() {
18         // 方式二：Lambda格式的按条件查询，可以防止字段名写错
19         QueryWrapper<User> qw = new QueryWrapper<>();
20         qw.lambda().lt(User::getAge, 10);
21         List<User> users = userDao.selectList(qw);
22         System.out.println(users);
23     }
24
25     @Test
26     void test3() {
27         // 方式三：Lambda格式的按条件查询——简化版
28         LambdaQueryWrapper<User> lqw = new LambdaQueryWrapper<>();
29         lqw.ge(User::getAge, 25);
30         List<User> users = userDao.selectList(lqw);
31         System.out.println(users);
32     }
33 }
```


1.2 多条件查询（支持链式编程）

```
1  @Test
2  void test4() {
3      LambdaQueryWrapper<User> lqw = new LambdaQueryWrapper<>();
4
5      // age>18 && age<22
6      // lqw.gt(User::getAge,18).lt(User::getAge,22);
7      // age<10 || age>30
8      lqw.lt(User::getAge, 10).or().gt(User::getAge, 30);
9
10     List<User> users = userDao.selectList(lqw);
11     System.out.println(users);
12 }
```

1.3 NULL值处理

为什么要处理null值？来看下面的业务案例：



前端页面提示用户输入目标商品的下限价格和上限价格，但有些情况下用户可能只输入下限值或上限值或两个都不输入，这样就会导致前端传到后端的上限值或下限值为null，这时候就需要对这些null值做处理。

```
1  @Test
2  void test5() {
```

```

3      // 模拟前端页面传递过来的查询数据
4      UserQuery userQuery = new UserQuery();
5      userQuery.setAge(18);
6      userQuery.setMaxAge(22);
7
8      // null值处理
9      LambdaQueryWrapper<User> lqw = new LambdaQueryWrapper<>();
10     /*
11         先判断第一个入参是否为true，如果为true就连接当前条件
12         等价于：
13         if (userQuery.getAge() != null) {
14             lqw.gt(User::getAge, userQuery.getMaxAge());
15         }
16     */
17     lqw.gt(userQuery.getAge() != null, User::getAge,
18 userQuery.getAge());
19     lqw.lt(userQuery.getMaxAge() != null, User::getAge,
20 userQuery.getMaxAge());
21
22     List<User> users = userDao.selectList(lqw);
23     System.out.println(users);
24 }

```

2.查询投影

💡 Tip

从查询结果中筛选并展示指定列（字段）的过程。

2.1 查询结果包含实体类中部分属性

```
1 @Test
2 // 查询投影
3 void test6(){
4     LambdaQueryWrapper<User> lqw = new LambdaQueryWrapper<>();
5     lqw.select(User::getId, User::getName, User::getAge);
6     List<User> users = userDao.selectList(lqw);
7     System.out.println(users);
8 }
```

2.2 查询结果包含聚合函数

```
1 @Test
2 void test7(){
3     QueryWrapper<User> lqw = new QueryWrapper<>();
4     lqw.select("count(*) as count, tel");
5     lqw.groupBy("tel");
6     List<Map<String, Object>> users=userDao.selectMaps(lqw);
7     System.out.println(users);
8 }
```

2.3 假如2.2不用MP，而用MyBatis+原生SQL

首先在 `UserDao` 接口中编写 `getTelGroup()` 这个接口方法：

```
1 @Mapper
2 public interface UserDao extends BaseMapper<User> {
3
4     @Select("select count(*) as count, tel from user group by tel")
5     List<Map<String, User>> getTelGroup();
6 }
```

其次在测试类中编写测试方法：

```
1 | @Test
2 | void test7Plus(){
3 |     List<Map<String, User>> telGroup = userDao.getTelGroup();
4 |     System.out.println(telGroup);
5 | }
```

可以看到，对于复杂、固定的查询，尤其涉及多表连接和复杂聚合时，使用MyBatis编写原生SQL可能是更好的选择。

3.查询条件设定

函数名	作用	等效SQL片段
<code>eq(R column, Object val)</code>	等于 (=)	<code>id = 10</code>
<code>ne(R column, Object val)</code>	不等于 (<>)	<code>name <> '张三'</code>
<code>gt(R column, Object val)</code>	大于 (>)	<code>age > 18</code>
<code>ge(R column, Object val)</code>	大于等于 (>=)	<code>age >= 18</code>
<code>lt(R column, Object val)</code>	小于 (<)	<code>age < 30</code>
<code>le(R column, Object val)</code>	小于等于 (<=)	<code>age <= 30</code>
<code>between(R column, Object val1, Object val2)</code>	区间匹配	<code>age between 18 and 30</code>
<code>like(R column, Object val)</code>	模糊匹配	<code>name like '%张%'</code>
<code>likeLeft(R column, Object val)</code>	左模糊匹配	<code>name like '%三'</code>
<code>likeRight(R column, Object val)</code>	右模糊匹配	<code>name like '张%'</code>
<code>isNull(R column)</code>	字段为空	<code>email is null</code>
<code>isNotNull(R column)</code>	字段不为空	<code>email IS not nu;;</code>
<code>orderByAsc(R... columns)</code>	升序排序 (ORDER BY column ASC)	<code>order by age ASC, id ASC</code>
<code>orderByDesc(R... columns)</code>	降序排序 (ORDER BY column DESC)	<code>order by age DESC, id DESC</code>
...	...	...

4. 字段映射与表名映射——@TableField + @TableName

4.1 表字段与实体类属性设计不同步

```
CREATE TABLE `user` (  
  `id` bigint(20) NOT NULL AUTO_INCREMENT,  
  `name` varchar(32),  
  `pwd` varchar(32),  
  `age` int(3),  
  `tel` varchar(32),  
  PRIMARY KEY (`id`)  
)
```

```
public class User {  
    private Long id;  
    private String name;  
    private String password;  
    private Integer age;  
    private String tel;  
}
```

解决方案：使用 @TableField 注解将实体类属性与表字段关联起来。

```
1 public class User {  
2     @TableField(value="pwd")  
3     private String password;  
4 }
```

4.2 实体类中添加了表中未定义的字段

```
CREATE TABLE `user` (  
  `id` bigint(20) NOT NULL AUTO_INCREMENT,  
  `name` varchar(32),  
  `pwd` varchar(32),  
  `age` int(3),  
  `tel` varchar(32),  
  PRIMARY KEY (`id`)  
)
```

```
public class User {  
    private Long id;  
    private String name;  
    @TableField(value="pwd")  
    private String password;  
    private Integer age;  
    private String tel;  
    private Integer online;  
}
```

解决方案：使用 @TableField 注解表明该属性在表中没有对应字段。

```

1 public class User {
2     @TableField(exist=false)
3     private Integer online;
4 }

```

注意： `exist` 无法与 `value` 同时使用。

4.3 采用 `select *` 查询开放了过多的字段查看权限，存在安全隐患

```
SELECT id, name, pwd, age, tel, speciality FROM user
```

```

CREATE TABLE `user` (
  `id` bigint(20) NOT NULL AUTO_INCREMENT,
  `name` varchar(32),
  `pwd` varchar(32),
  `age` int(3),
  `tel` varchar(32),
  PRIMARY KEY (`id`)
)

```

```

public class User {
    private Long id;
    private String name;
    @TableField(value="pwd")
    private String password;
    private Integer age;
    private String tel;
    @TableField(exist = false)
    private Integer online;
}

```

解决方案：使用 `@TableField` 注解设置某个表字段不可查询。

```

1 public class User {
2     @TableField(value="pwd", select=false)
3     private String password;
4 }

```

4.4 表名与实体类名设计不同步

```
CREATE TABLE `tbl_user` (  
  `id` bigint(20) NOT NULL AUTO_INCREMENT,  
  `name` varchar(32),  
  `pwd` varchar(32),  
  `age` int(3),  
  `tel` varchar(32),  
  PRIMARY KEY (`id`)  
)
```

```
public class User {  
    private Long id;  
    private String name;  
    @TableField(value="pwd",select = false)  
    private String password;  
    private Integer age;  
    private String tel;  
    @TableField(exist = false)  
    private Integer online;  
}
```

解决方案：使用 `@TableName` 注解将实体类与表关联起来。

```
1 | @TableName("tbl_user")  
2 | public class User {  
3 |     ...  
4 | }
```

四、DML控制

五、快速开发

